

App Dev Project -1 Report

1. Student Details

Name: Narukulla Akhila

Roll Number:24F2001995

Email: 24f2001995@ds.study.iitm.ac.in

About Me: I am an undergraduate Computer Science student pursuing a B.S. Degree in Data Science and Applications from IIT Madras alongside a B.Tech in Computer Science and Engineering (Artificial Intelligence & Machine Learning). I enjoy coding, solving programming problems, and learning about Artificial Intelligence, Machine Learning, and Data Science.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

To design and build a web-based Trekking Management Application that allows administrators to manage treks, staff, and users, staff to manage assigned treks, and users to browse, book, and track trekking activities.

Approach:

The application was built using Flask as the backend framework with a modular structure. It allows administrators to manage treks and staff, staff members to manage assigned treks and participants, and users to browse, search, filter, and book treks through separate role-based dashboards.

3. AI/LLM Declaration

I used ChatGPT (GPT-5.5) to assist with understanding Flask concepts, debugging, code suggestions, improving HTML templates, and preparing project documentation. The extent of AI/LLM usage is approximately 25%. All final implementation, integration, testing, and validation were carried out by me.

4. Technologies and Frameworks Used

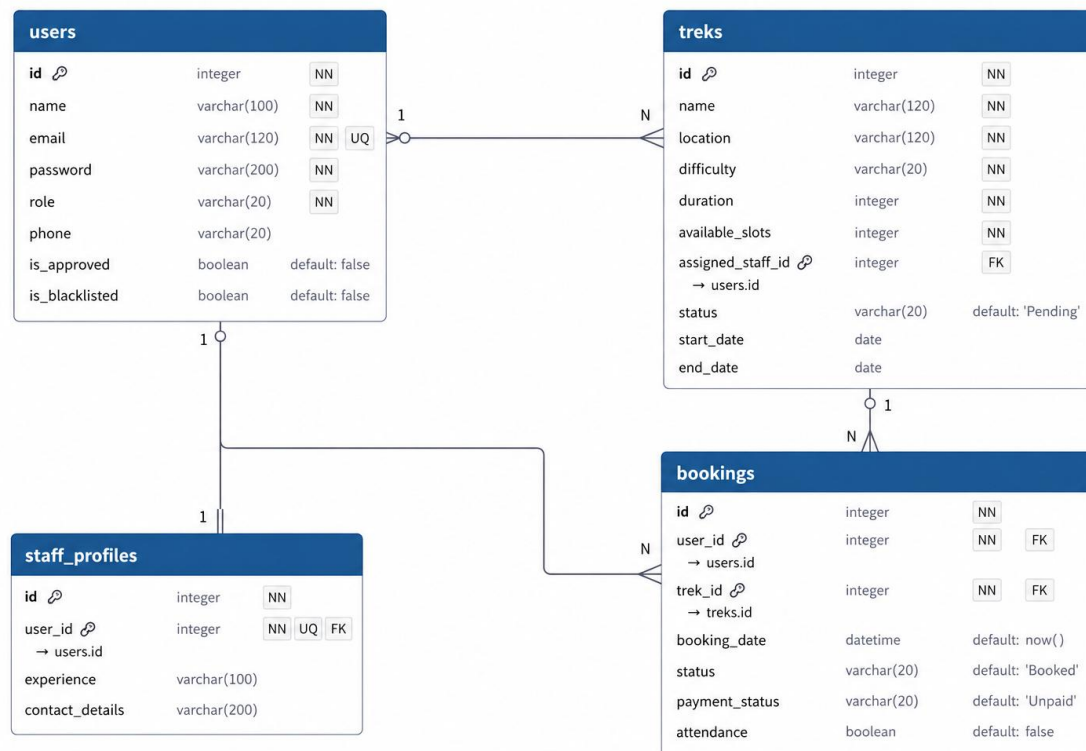
Technology / Library	Purpose
Flask	Core Python web framework used to build the application.

Flask-SQLAlchemy	ORM used to define models and interact with the SQLite database.
SQLite	Lightweight relational database used to store users, treks, bookings, and staff information.
Jinja2	Template engine used to render dynamic HTML pages with data from Flask.
HTML5	Structure of the web pages and forms.
CSS3	Custom styling for the application interface.
Bootstrap 5	Responsive layout, navigation bar, tables, forms, buttons, and cards.
Python	Backend programming language for implementing application logic.

5. Database Schema / ER Diagram

Table	Columns	Constraints / Description
User	id, name, email, password, role, phone, is_approved, is_blacklisted	id – Primary Key, email – Unique, name, email, password, role – NOT NULL
Trek	id, name, location, difficulty, duration, available_slots, assigned_staff_id, status, start_date, end_date	id – Primary Key, assigned_staff_id – Foreign Key → users.id, mandatory fields are NOT NULL
Booking	id, user_id, trek_id, booking_date, status, payment_status, attendance	id – Primary Key, user_id – Foreign Key → users.id, trek_id – Foreign Key → treks.id, booking date stored automatically
StaffProfile	id, user_id, experience, contact_details	id – Primary Key, user_id – Foreign Key → users.id

ER diagram: -



6. API Resource Endpoints

REST APIs were not implemented in this project.

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – main Flask application entry point
- **/models** – database models using SQLAlchemy
- **/routes** – Flask Blueprints for user and activity routes
- **/templates** – Jinja2 HTML templates
- **/static** – CSS

Implemented Features

- User, Staff, and Admin authentication.
- Role-based dashboards.
- Trek creation, editing, deletion, and staff assignment.
- Trek booking with overbooking prevention.

- Trek search and filtering by location and difficulty.
- Booking history, payment status, and cancellation management.
- Responsive user interface built using Bootstrap.

Additional Features

- User and Staff profile editing.
 - Automatic slot restoration after booking cancellation.
 - View and manage participants for Admin and assigned Staff.
 - Dashboard statistics for all user roles.
 - User and Staff blacklisting/unblacklisting.
 - Search by both Name and ID.
 - Attendance marking by staff for a trek assigned to them.
-

8. Video Presentation

Drive Link: